Bank Telemarketing Predictive Analysis

Introduction

For our analysis, our group used predictive modeling to evaluate the effectiveness of telemarketing campaigns in encouraging clients to subscribe to bank term deposits. In essence, a bank term deposit is a commitment to setting aside a specific amount of money for a set period, allowing this sum of money to collect interest over that time. Our dataset is sourced from the UC Irvine Machine Learning Repository, where the data itself is from a Portuguese banking institution concerned with the effectiveness of direct marketing campaigns. We believe that predictive modeling can serve as a powerful tool in identifying inefficiencies within marketing strategies. Our goal is to conduct a thorough exploration of our selected dataset in order to identify key factors that drive client subscriptions as well as evaluate the effectiveness of these telemarketing campaigns overall.

The dataset contains 45,211 observations, and 17 variables (16 input + 1 output):

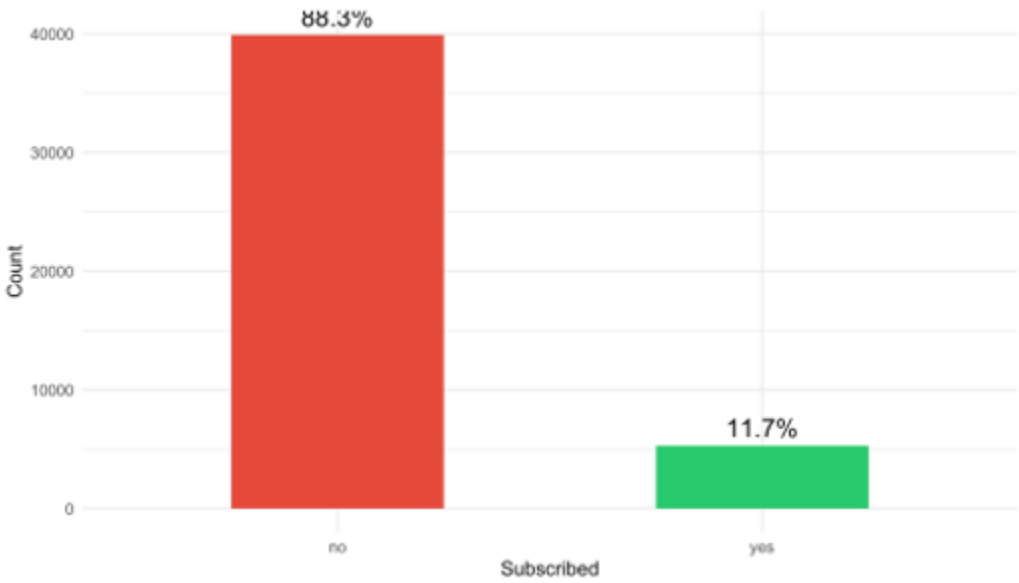
- Client Demographics: age, job, marital status, default, balance, housing, loan
- Campaign Contact Info: contact, day, month, duration, campaign
- Previous Campaign Info: pdays, previous, poutcome
- **Output:** y – term deposit subscription: yes/no

Given this, our central business question is:

Can we predict if a client will subscribe to a term deposit based on these variables?

Problem Description/Initial Exploration

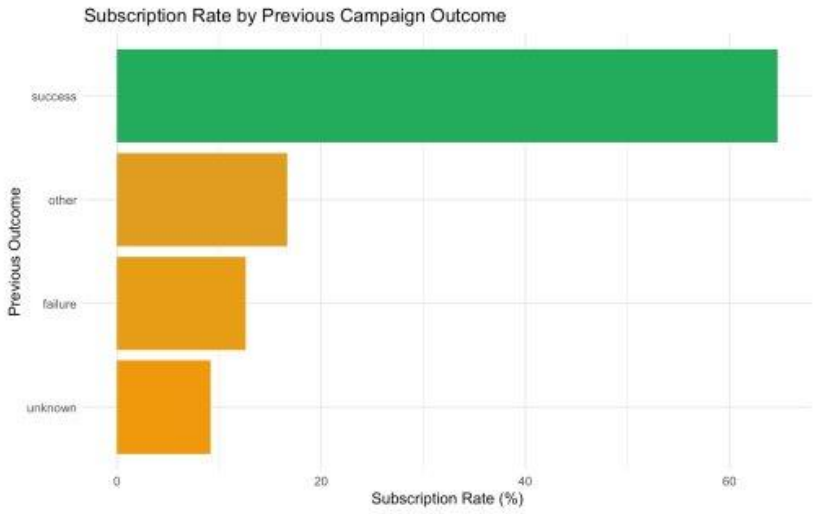
In our dataset, we have 16 input features, 1 output feature, and 45,211 total observations. Our output feature, otherwise referred to as our 'target variable,' is labeled as 'subscribed,' where we only have two possible outcomes; 'yes' or 'no.' Upon performing basic exploration of our data, we can observe that, generally speaking, telemarketing campaigns are an extremely inefficient marketing practice. Shown below, we have a figure illustrating the difference between whether a client subscribes or not:



Perhaps unsurprisingly, the difference between 'yes' and 'no' is stark, with only 11.7% of our total records showing a client subscribed as a result of this marketing strategy. This provides a key insight with regards to our accuracy, sensitivity, and specificity measures we will calculate later on using our predictive models. This fact connects to the big issue of imbalanced datasets. Because the vast majority of our observations fall into the 'no' category, our model could obtain a high degree of overall accuracy by simply predicting 'no' for each observation. In relation to sensitivity and specificity measures, we would expect utilizing such a highly imbalanced dataset to yield a very high specificity percentage, as well as a sensitivity percentage much lower than

that of the calculated overall accuracy and specificity. In simpler terms, since our dataset is so heavily skewed in favor of the 'no' outcomes, this suggests that our models utilizing this data will be much better at correctly identifying cases in which the observation of 'subscribed' resulted in 'no,' while the opposite is true when it comes to predicting observations that resulted in 'yes.'

While our overall results are heavily skewed in favor of 'no,' an interesting observation we identified within our initial data exploration was that subscription rates are strongly influenced by the previous telemarketing campaign outcome. Meaning, a client that previously subscribed was significantly more likely to subscribe once again during another marketing campaign. Shown below, we have a figure illustrating this observation.



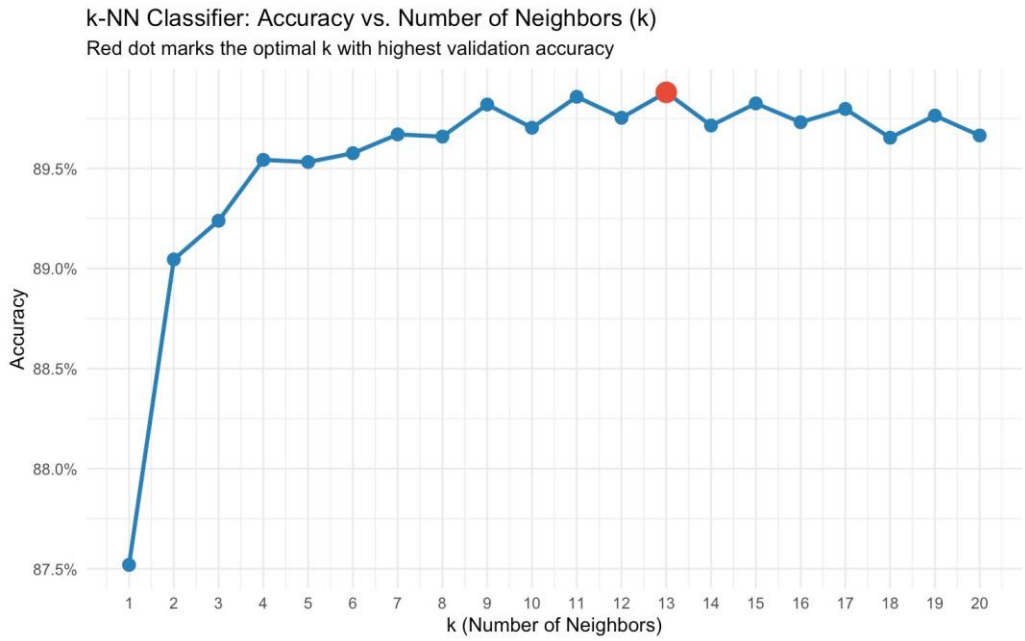
Though it may not be surprising, this is still an interesting observation since it demonstrates the strength of using a previous subscription from a specific client as a predictor. From our figure we observe that for a specific client, if the previous campaign outcome was a success, the subscription rate for these individuals in a following marketing campaign was approximately 65%, while those who did not previously subscribe are much less likely to subscribe during a future campaign. This information strongly suggests that a client's previous

behavior serves as a somewhat reliable indicator of whether or not they will subscribe in the future, providing evidence that this may be a valuable predictor in a model. While these are interesting observations, our predictive models will serve to provide deeper analysis incorporating all of our variables and their total impact on subscription outcomes together.

Approach

In terms of modeling approaches, we ultimately went with classification modeling over regression modeling. Since we are concerned with measuring categorical outcomes (yes/no) rather than continuous numeric outcomes, the classification approach is the more logical option compared to regression. Classification models are generally utilized to divide data points into class groups, and since we want to predict whether a client will subscribe to a term deposit or not, this approach is clearly better suited to accommodate our binary target variable. We utilized three classification models in particular:

- K-Nearest Neighbors (K-NN), logistic regression, and a classification tree.



The 3 classification models we used each have strengths and weaknesses. K-NN is a non-parametric method that classifies based on the characteristics of nearby data points and requires no assumptions about the underlying distribution. Logistic regression is also a parametric model that estimates the probability of subscription as a function of the input variables, with the added benefit of coefficients that quantify each features' effect on the outcome. Finally, our classification tree partitions the data through a series of binary decision rules, producing a visual and intuitive model that easily explains the classifying process.

For our test/train split, we decided to utilize a 60/40 split between our training and validation data. We tested other splits such as 70/30 and 80/20, but we found that this did not have a substantial impact on our computed results. Although unconventional, our initial logic behind allocating more data for our test set was that since our target variable was so skewed, it might be better to ensure we have a larger sample of our 'yes' observations within the target variable.

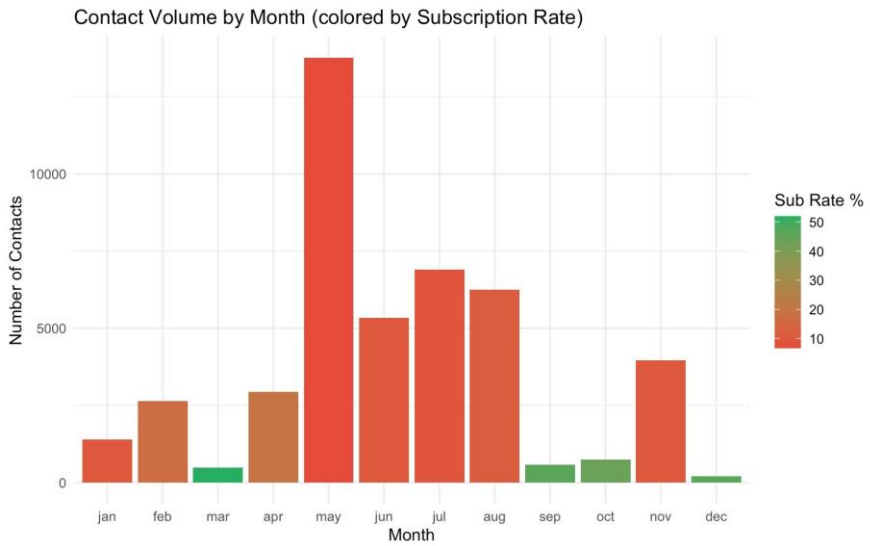
Data Analysis

Our immediate understanding when looking at the dataset was that our target variable was heavily skewed with 88% of clients not subscribing ("no"), leaving only around 11% saying yes. This context is important when evaluating model performance since saying no only yields an 88% accuracy.

Key Findings

- Call duration showed the strongest separation between subscribers and non-subscribers. Those who subscribed had substantially longer calls on average

- Campaign contacts are slightly lower for subscribers, and over-contacting is associated with reduced conversion
- Account balance and age show modest differences between groups but are not strongly strong correlators on their own



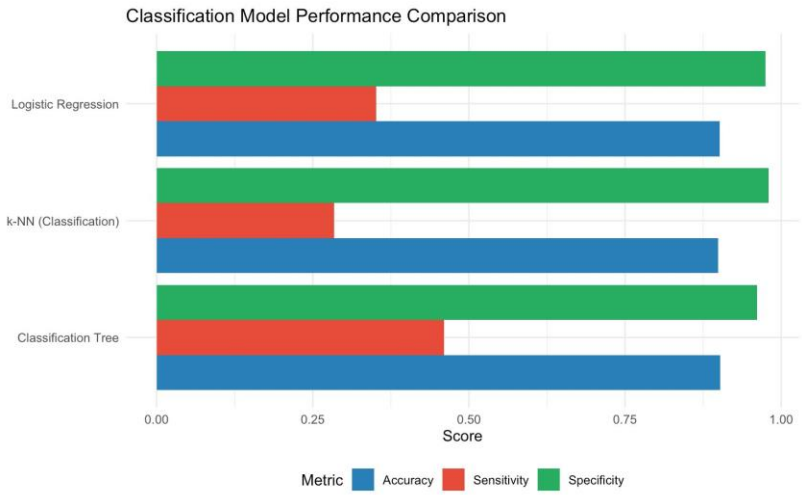
Furthermore, categorical variables by subscription rate revealed that clients with a prior successful campaign outcome (poutcome = success), subscribed at ~65%, almost 6x the baseline of 11.7%. Interestingly, students and retired clients were the highest subscription rates by job type, and blue-collar workers showed the lowest rates. March, September, October, and December showed high conversion rates, with May having the highest contact volume with below average rates. We believe the inclusion of month information hindered the performance of our model, as contacts weren't consistent month-by-month.

Results/Inference

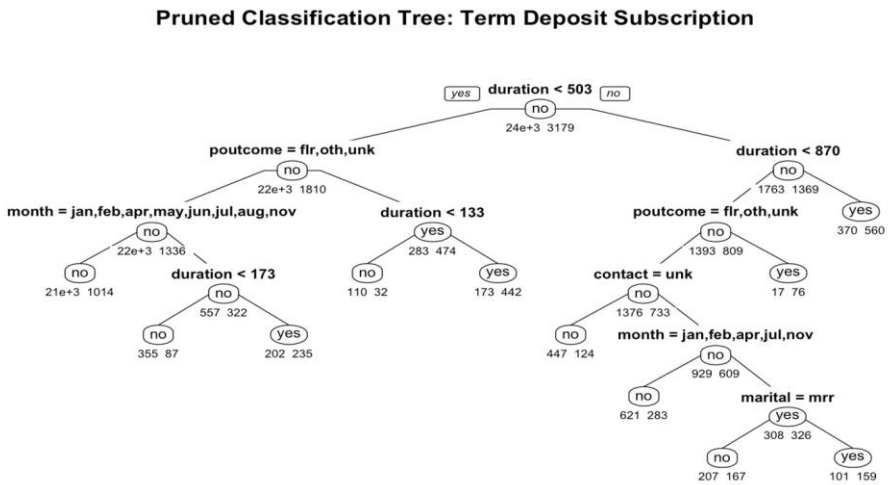
Model	Accuracy	Sensitivity	Specificity
k-NN	89.8%	28.4%	98%

Logistic Regression	90%	35.1%	97.4%
Classification Tree	90.3%	46%	96.1%

All models were evaluated on the same validation set. Above is a table summarizing the accuracy, sensitivity (ability to correctly identify subscribers), and specificity (ability to correctly identify non-subscribers).



The model achieved similar overall accuracy, ~90%, a potential reflection of the 88% non-subscriber rate. Sensitivity is substantially more difficult to get right due to the imbalance of our target variable. Below is our final classification tree.



Conclusion & Recommendations

After analyzing the data with multiple models, we've concluded that prioritizing clients with prior success is a smart strategy to increase signups. This was the main node in our classification tree which alludes to the strength of this predictor. On the flip side of that also, cap outreach at 2-3 contacts per campaign cycle as the data shows additional calls still unsuccesful.

Insights were found; however, there is also some context to better understand the information gained from the data. Firstly, we believe that the model finding correlation with campaign timing is potentially erroneous due to the presence of inconsistent data by month. Further, with such a strong class imbalance between yes and no (~88% no), overall accuracy makes a misleading metric. Future work should focus on improving the sensitivity of the models.

Determining which specific model would be most ideal by analyzing our table results, it seems that a classification tree would be the most ideal model of our three. Although sensitivity remains low relative to our overall accuracy and specificity within the model, we still observe a notable increase in this metric compared to our k-NN and logistic regression models. Additionally, our overall accuracy is highest when utilizing our classification tree, while the decrease we observe in specificity is fairly insignificant. Overall, this suggests that a classification tree is the most balanced model we could utilize, providing us greater ability to predict what drives clients to subscribe without meaningfully compromising overall accuracy or specificity.

Code from RMD file:

Data Loading and Preparation

```
rm(list = ls())

library("tidyverse")
library("FNN")
library("rpart")
library("rpart.plot")
library("caret")

bank = read_delim("bank-full.csv", delim = ";", col_types = cols(.default = "c"))


# convert numeric columns to numeric type

bank = bank %>%
  mutate(across(c(age, balance, day, duration, campaign, pdays, previous), as.numeric))
```

Data Overview

```
glimpse(bank)

# number of rows and columns

nrow(bank)
ncol(bank)
```

Data Quality Check

```
# check for missing values

colSums(is.na(bank))
```

```
# target variable distribution

bank %>%
  group_by(y) %>%
  summarise(count = n()) %>%
  mutate(proportion = round(count / sum(count), 4))
```

The dataset has **no missing values**. The target variable is imbalanced: approximately **88.3% "no"** and **11.7% "yes"**.

Data Preparation

```
bank <- bank %>%
  rename(y_actual = y) %>%
  mutate(
    y_actual = factor(y_actual, levels = c("no", "yes")),
    job = factor(job),
    marital = factor(marital),
    education = factor(education, levels = c("primary", "secondary", "tertiary", "unknown")),
    default = factor(default),
    housing = factor(housing),
    loan = factor(loan),
    contact = factor(contact),
    month = factor(month, levels = c("jan","feb","mar","apr","may","jun",
"jul","aug","sep","oct","nov","dec")),
    poutcome  = factor(poutcome),
    id = row_number()
  )
```

Dummy Variables (for k-NN)

```
bank_dummies <- model.matrix(y_actual ~ age + job + marital + education + default + balance +
housing + loan + contact + day + month +
duration + campaign + pdays + previous + poutcome,
data = bank) %>%
```

```
  as_tibble() %>%
  select(-`(Intercept)`) %>%
  mutate(y_actual = bank$y_actual,id = bank$id)
```

Exploratory Data Analysis

Target Variable Distribution

```
# bar chart of subscription outcome

bank %>%
  group_by(y_actual) %>%
  summarise(count = n()) %>%
  mutate(pct = round(count / sum(count) * 100, 1)) %>%
  ggplot(aes(x = y_actual, y = count, fill = y_actual)) +
  geom_bar(stat = "identity", width = 0.5) +
  geom_text(aes(label = paste0(pct, "%")), vjust = -0.5, size = 5) +
  scale_fill_manual(values = c("no" = "red", "yes" = "green")) +
  labs(title = "Term Deposit Subscription Distribution",
       subtitle = "Significant class imbalance: ~88% did not subscribe",
       x = "Subscribed", y = "Count") +
  theme_minimal() +
  theme(legend.position = "none")
```

Numeric Variables by Subscription Outcome

```
# boxplots of numeric input variables grouped by subscription outcome

p1 <- ggplot(bank, aes(x = y_actual, y = age, fill = y_actual)) +
  geom_boxplot() + labs(title = "Age", x = NULL, y = NULL) +
  scale_fill_manual(values = c("no" = "red", "yes" = "green")) +
  theme_minimal() + theme(legend.position = "none")

p2 <- ggplot(bank, aes(x = y_actual, y = balance, fill = y_actual)) +
```

```
  geom_boxplot() + labs(title = "Balance", x = NULL, y = NULL) +
  scale_fill_manual(values = c("no" = "red", "yes" = "green")) +
  theme_minimal() + theme(legend.position = "none")

p3 <- ggplot(bank, aes(x = y_actual, y = duration, fill = y_actual)) +
  geom_boxplot() + labs(title = "Call Duration (seconds)", x = NULL, y = NULL) +
  scale_fill_manual(values = c("no" = "red", "yes" = "green")) +
  theme_minimal() + theme(legend.position = "none")

p4 <- ggplot(bank, aes(x = y_actual, y = campaign, fill = y_actual)) +
  geom_boxplot() + labs(title = "Campaign Contacts", x = NULL, y = NULL) +
  scale_fill_manual(values = c("no" = "red", "yes" = "green")) +
  theme_minimal() + theme(legend.position = "none")

gridExtra::grid.arrange(p1, p2, p3, p4, ncol = 2)
```

Categorical Variables by Subscription Rate

```
# subscription rate by job type

bank %>%
  group_by(job) %>%
  summarise(sub_rate = mean(y_actual == "yes") * 100) %>%
  arrange(desc(sub_rate)) %>%
  ggplot(aes(x = reorder(job, sub_rate), y = sub_rate, fill = sub_rate)) +
  geom_bar(stat = "identity") +
  coord_flip() +
  scale_fill_gradient(low = "orange", high = "green") +
  labs(title = "Subscription Rate by Job Type", x = "Job", y = "Subscription Rate (%)") +
  theme_minimal() + theme(legend.position = "none")

# subscription rate by previous campaign outcome

bank %>%
  group_by(poutcome) %>%
  summarise(sub_rate = mean(y_actual == "yes") * 100) %>%
  ggplot(aes(x = reorder(poutcome, sub_rate), y = sub_rate, fill = sub_rate)) +
  geom_bar(stat = "identity") +
  coord_flip() +
```

```
  scale_fill_gradient(low = "orange", high = "green") +
  labs(title = "Subscription Rate by Previous Campaign Outcome",
       x = "Previous Outcome", y = "Subscription Rate (%)") +
  theme_minimal() + theme(legend.position = "none")
```

Monthly Contact Volume and Subscription Rate

```
# contact volume by month, colored by subscription rate

bank %>%
  group_by(month) %>%
  summarise(n_contacts = n(),
            sub_rate   = mean(y_actual == "yes") * 100) %>%
  ggplot(aes(x = month, y = n_contacts, fill = sub_rate)) +
  geom_bar(stat = "identity") +
  scale_fill_gradient(low = "red", high = "green", name = "Sub Rate %") +
  labs(title = "Contact Volume by Month (colored by Subscription Rate)",
       x = "Month", y = "Number of Contacts") +
  theme_minimal()
```

Train / Validation Split

```
set.seed(30)


# 60/40 train validation split

train <- bank %>% sample_frac(0.6)
validation <- bank %>% slice(setdiff(bank$id, train$id))

# split for dummy variables
train_dum <- bank_dummies %>% filter(id %in% train$id)
validation_dum <- bank_dummies %>% filter(id %in% validation$id)
```

Classification Models

k-NN as Classifier

Step 1: Standardize Numeric Input Variables Using Train Data Statistics

```
numeric_cols <- c("age", "balance", "day", "duration", "campaign", "pdays", "previous")

#standardize
train_means <- train_dum %>% select(all_of(numeric_cols)) %>%
summarise(across(everything(), mean))
train_sds <- train_dum %>% select(all_of(numeric_cols)) %>% summarise(across(everything(),
sd))

for (col in numeric_cols) {
  train_dum[[paste0(col, "_norm")]] <- (train_dum[[col]] - train_means[[col]]) / train_sds[[col]]
validation_dum[[paste0(col, "_norm")]] = (validation_dum[[col]] - train_means[[col]]) /
train_sds[[col]]
}
```

Step 2: Pick Standardized Input Variables

```
dummy_cols <- setdiff(names(train_dum), c(numeric_cols, paste0(numeric_cols, "_norm"),
"y_actual", "id"))
train_input_norm <- train_dum %>%
  select(all_of(paste0(numeric_cols, "_norm")), all_of(dummy_cols))
validation_input_norm = validation_dum %>%
  select(all_of(paste0(numeric_cols, "_norm")), all_of(dummy_cols))
```

Step 3: Find the Optimal k

```
# loop through k = 1 to 20 and record accuracy for each k

options(warn = -1)
knn_accuracy = c()
```

```
for (k in 1:20) {
  preds <- knn(train_input_norm, validation_input_norm, train_dum$y_actual, k)
  knn_accuracy[k] <- confusionMatrix(factor(preds, levels=c("no","yes")),
validation_dum$y_actual)$overall[1]
}

best_k <- which.max(knn_accuracy)
cat("Optimal k is:", best_k, "\n")
```

Step 4: Visualize Accuracy Across k Values

```
#data frame of k values and their corresponding accuracy

knn_results <- tibble(k = 1:20, accuracy = knn_accuracy)
best_k <- which.max(knn_accuracy)

cat("Best k:", best_k, "with accuracy:", round(max(knn_accuracy), 4), "\n")


# plot accuracy vs k

ggplot(knn_results, aes(x = k, y = accuracy)) +
  geom_line(color = "steelblue", size = 1.1) +
  geom_point(color = "steelblue", size = 3) +
  geom_point(data = filter(knn_results, k == best_k),
             aes(x = k, y = accuracy),
             color = "red", size = 5) +
  geom_text(data = filter(knn_results, k == best_k),
            aes(x = k, y = accuracy,
                label = paste0("Best k = ", best_k, "\n", round(accuracy, 4))),
            vjust = -1.2, color = "red", size = 4, fontface = "bold") +
  scale_x_continuous(breaks = 1:20) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 0.1)) +
  labs(title = "k-NN Classifier: Accuracy vs. Number of Neighbors (k)",
       subtitle = "Red dot marks the optimal k with highest validation accuracy",
       x = "k (Number of Neighbors)", y = "Accuracy") +
  theme_minimal()
```

Step 5: Predict Using Best k and Evaluate

```
knn_pred <- knn(train_input_norm, validation_input_norm, train_dum$y_actual, best_k)
validation <- validation %>%
  mutate(y_prediction = factor(knn_pred, levels = c("no", "yes")))

# confusion matrix, accuracy, sensitivity & specificity

confusionMatrix(validation$y_prediction, validation$y_actual, positive = "yes")
```

Logistic Regression as Classifier

```
# fit logistic regression on training data

bank.lr <- glm(y_actual ~ age + job + marital + education + default + balance + housing + loan +
contact + day + month + duration + campaign + pdays + previous + poutcome,
            data   = train, family = binomial(link = "logit"))

summary(bank.lr)

lr_probs <- predict(bank.lr, newdata = validation, type = "response")

validation <- validation %>%
  mutate(y_prediction_lr = factor(if_else(lr_probs > 0.5, "yes", "no"),
                                  levels = c("no", "yes")))

confusionMatrix(validation$y_prediction_lr, validation$y_actual, positive = "yes")

lr_coefs <- summary(bank.lr)$coefficients

sig_coefs <- as_tibble(lr_coefs, rownames = "term") %>%
  rename(estimate = Estimate, p_value = `Pr(>|z|)`) %>%
  filter(p_value < 0.05, term != "(Intercept)") %>%
  arrange(estimate)

ggplot(sig_coefs, aes(x = reorder(term, estimate), y = estimate, fill = estimate > 0)) +
  geom_bar(stat = "identity") +
```

```
  coord_flip() +
  scale_fill_manual(values = c("TRUE" = "green", "FALSE" = "red"),
                    labels = c("Decreases probability", "Increases probability"),
                    name = "") +
  labs(title = "Logistic Regression: Significant Predictors (p < 0.05)",
       x = "Feature", y = "Log-Odds Estimate") +
  theme_minimal()
```

Classification Tree

Step 1: Build the Tree on Training Data + Prune

```
# fit a classification tree on the training data
# use cp = 0.00001, minsplit = 10, xval = 10
# assign it to an object "bank.ct"

bank_ct <- rpart(y_actual ~ age + job + marital + education + default + balance + housing + loan
+ contact + day + month + duration + campaign +   pdays + previous + poutcome,
                 data     = train,
                 method   = "class",
                 cp       = 0.00001,
                 minsplit = 10,
                 xval     = 10)
#prune
cp_table <- as.data.frame(bank_ct$cptable)
optimal_cp <- cp_table %>% filter(nsplit == 10) %>% pull(CP)

pruned_ct <- prune(bank_ct, cp = optimal_cp)
```

Step 2: Plot

```
prp(pruned_ct, type = 1, extra = 1, under = TRUE, split.font = 2, varlen = -10)

cp_table <- as_tibble(bank_ct$cptable)
cp_table
```

Step 3: Predict on Validation Data

```
# predict subscription status

ct_results <- predict(pruned_ct, validation, type = "class")

validation <- validation %>%
  mutate(y_prediction_ct = factor(ct_results, levels = c("no", "yes")))

# confusion matrix, accuracy, sensitivity & specificity

confusionMatrix(validation$y_prediction_ct, validation$y_actual, positive = "yes")
```

Model Performance Comparison Data

```
# compute confusion matrix for each model and extract key metrics

cm_knn <- confusionMatrix(validation$y_prediction, validation$y_actual, positive = "yes")
cm_lr  <- confusionMatrix(validation$y_prediction_lr, validation$y_actual, positive = "yes")
cm_ct  <- confusionMatrix(validation$y_prediction_ct, validation$y_actual, positive = "yes")

model_comparison <- tibble(
  Model       = c("k-NN (Classification)", "Logistic Regression", "Classification Tree"),
  Accuracy    = c(cm_knn$overall[1],  cm_lr$overall[1],  cm_ct$overall[1]),
  Sensitivity = c(cm_knn$byClass[1],  cm_lr$byClass[1],  cm_ct$byClass[1]),
  Specificity = c(cm_knn$byClass[2],  cm_lr$byClass[2],  cm_ct$byClass[2])
) %>%
  mutate(across(where(is.numeric), ~ round(.x, 4)))

model_comparison
```

Comparison Visusal

```
model_long = model_comparison %>%
  pivot_longer(cols = c(Accuracy, Sensitivity, Specificity),
               names_to  = "Metric",
               values_to = "Value")



# grouped bar chart comparing all three models


ggplot(model_long, aes(x = Model, y = Value, fill = Metric)) +
  geom_bar(stat = "identity", position = "dodge") +
  coord_flip() +
  scale_fill_manual(values = c("Accuracy"    = "steelblue",
                               "Sensitivity" = "red",
                               "Specificity" = "green")) +
  labs(title = "Classification Model Performance Comparison",
       x = NULL, y = "Score") +
  theme_minimal() +
  theme(legend.position = "bottom")
```